Contents

1. Dynamic Programming

This chapter covers Dynamic Programming (DP) problems. For a comprehensive introduction to DP concepts, memoization, and tabulation, see the **Dynamic Programming** section in Concepts.

1.1. Dice Combinations

Question - Dice Combinations Backup Link

Explanation :

There is exactly one way to get a 0, by not throwing. Now, to reach any sum, we imagine the last move we took. If the last move was `dice`, then we must have already reached `sum - dice`. All the ways to reach `sum - dice` automatically become valid ways to reach sum. By adding these possibilities for all dice values from 1 to 6, we grow the solution from smaller sums to larger ones. This approach avoids repeated work and makes the counting process both efficient and intuitive, while the modulo simply prevents numbers from growing too large.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    // dp[i] = number of ways to reach sum i
    vector<int> dp(n + 1, 0);
    dp[0] = 1;  // Base case: one way to make sum 0

    for (int sum = 1; sum <= n; sum++) {
        for (int dice = 1; dice <= 6; dice++) {
            if (sum - dice >= 0) {
                dp[sum] = (dp[sum] + dp[sum - dice]) % MOD;
            }
        }
    }
```

```
25
26      cout << dp[n] << '\n';
27      return 0;
28  }
```

1.2. Minimizing Coins

Question - Minimizing Coins Backup Link

Explanation :

We are given `n` coin denominations and a target sum. Our goal is to find the minimum number of coins needed to form exactly that sum, where each coin can be used any number of times.

Key Observation: The greedy approach (always picking the largest coin) does not always work. For example, with coins `[1, 3, 4]` and target `6`, greedy gives `4 + 1 + 1 = 3` coins, but optimal is `3 + 3 = 2` coins.

Dynamic Programming Insight: Instead of exploring all combinations, we build the solution bottom-up. For any sum `x`, we form it by taking one coin `c` and adding it to the optimal solution for sum `x - c`. We try all possible coins and pick the best.

Building the Solution: Start with `dp[0] = 0` (zero coins for sum 0). For each sum from `1` to `target`, try every coin. If using coin `c` gives a better result, update `dp[sum] = min(dp[sum], dp[sum - c] + 1)`. Mark impossible sums with a large value.

Why This Works: When computing `dp[sum]`, all smaller values are already solved. By processing sums in increasing order, every subproblem is solved before we need it. This avoids recalculation and guarantees the optimal solution.

Code :

```C++
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, target;
    cin >> n >> target;

    vector<int> coins(n);
    for (int i = 0; i < n; i++) {
        cin >> coins[i];
    }

    // dp[i] = minimum number of coins needed to make sum i
    vector<int> dp(target + 1, INF);
    dp[0] = 0;
```

```
21
22      for (int sum = 1; sum <= target; sum++) {
23          for (int coin : coins) {
24              if (coin <= sum && dp[sum - coin] != INF) {
25                  dp[sum] = min(dp[sum], dp[sum - coin] + 1);
26              }
27          }
28      }
29
30      if (dp[target] == INF) {
31          cout << -1 << '\n';
32      }
33      else {
34          cout << dp[target] << '\n';
35      }
36
37      return 0;
38  }
```

1.3. Coin Combinations I

Question - Coin Combinations I Backup Link

Explanation :

We need to count the number of ordered ways to form a target sum using given coins. The key word here is "ordered" - this means `[1,2]` and `[2,1]` are considered different combinations.

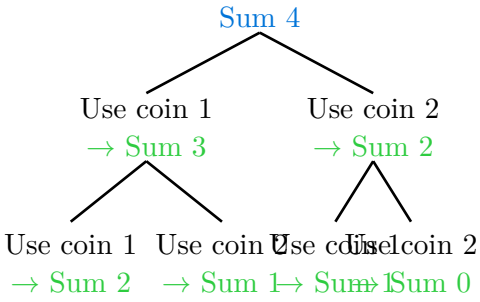
Key Insight: For any sum, we can reach it by adding any coin to a smaller sum. If we want sum `x`, we can:

- Take a coin of value 1 and add it to sum `x-1`
- Take a coin of value 2 and add it to sum `x-2`
- And so on for all coin values

Building the Solution: We start with `dp[0] = 1` (one way to make 0: use no coins). For each sum from 1 to target, we iterate through all coins. For each coin `c` that doesn't exceed the current sum, we add `dp[sum - c]` to `dp[sum]`. This counts all the ways we can form `sum - c` and then add coin `c` to reach our target sum.

Why Order Matters: Because we process each sum independently and consider all coins at each step, we naturally count all orderings. When we're at sum 5 with coins [1,2], we count paths like 1+1+1+1+1, 1+1+1+2, 1+1+2+1, 1+2+1+1, 2+1+1+1, etc.

Example: coins = [1, 2], target = 4



Each path to sum 0 is one valid combination!

Visual Example:

```
dp[0] = 1  (base case)
dp[1] = dp[1-1] + dp[1-2]
      = dp[0] + 0 = 1          → [1]

dp[2] = dp[2-1] + dp[2-2]
      = dp[1] + dp[0] = 2      → [1,1], [2]

dp[3] = dp[3-1] + dp[3-2]
      = dp[2] + dp[1] = 3      → [1,1,1], [1,2], [2,1]

```

```
dp[4] = dp[4-1] + dp[4-2]
      = dp[3] + dp[2] = 5      → [1,1,1,1], [1,1,2], [1,2,1], [2,1,1], [2,2]
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, target;
    cin >> n >> target;

    vector<int> coins(n);
    for (int i = 0; i < n; i++) {
        cin >> coins[i];
    }

    // dp[i] = number of ordered ways to form sum i
    vector<int> dp(target + 1, 0);
    dp[0] = 1;  // Base case: one way to make sum 0

    for (int sum = 1; sum <= target; sum++) {
        for (int coin : coins) {
            if (coin <= sum) {
                dp[sum] = (dp[sum] + dp[sum - coin]) % MOD;
            }
        }
    }

    cout << dp[target] << '\n';
    return 0;
}
```

1.4. Coin Combinations II

Question - Coin Combinations II Backup Link

Explanation :

This problem asks for the number of unordered ways to form a target sum. Unlike Coin Combinations I, here `[1,2]` and `[2,1]` are considered the same combination. We only count distinct sets of coins.

The Critical Difference: In Coin Combinations I, we processed sums and tried all coins at each sum. This naturally counted all orderings. Here, we need to avoid counting duplicates by processing coins in order and building up sums for each coin.

The Key Insight: Process coins one by one. For each coin, update all sums that can be formed by adding this coin. By processing coins in a fixed order, we ensure each combination is counted exactly once.

Algorithm:
1. Start with `dp[0] = 1` (one way to make 0)
2. For each coin in order:
   - For each sum from coin value to target:
     - Add ways to form `sum - coin` to `dp[sum]`
3. By the time we finish processing all coins, `dp[target]` contains the answer

Why This Avoids Duplicates: When we process coin 1, we count all combinations using only coin 1. When we process coin 2, we only add combinations that include at least one coin 2 (built on top of previous combinations). This ensures we never count the same set twice.

Example: coins = [1, 2], target = 4

Initially:
dp[0] = 1, rest = 0

After coin 1:
dp[0]=1, dp[1]=1, dp[2]=1
dp[3]=1, dp[4]=1
Combinations: [1], [1,1], [1,1,1], [1,1,1,1]

After coin 2:
dp[0]=1, dp[1]=1, dp[2]=2
dp[3]=2, dp[4]=3
New: [2], [1,2], [2,2]

Visual Trace for target = 4:

```
Initial: dp = [1, 0, 0, 0, 0]

Process coin 1:
  sum 1: dp[1] += dp[1-1] = dp[0] = 1  →  [1, 1, 0, 0, 0]
  sum 2: dp[2] += dp[2-1] = dp[1] = 1  →  [1, 1, 1, 0, 0]
  sum 3: dp[3] += dp[3-1] = dp[2] = 1  →  [1, 1, 1, 1, 0]
  sum 4: dp[4] += dp[4-1] = dp[3] = 1  →  [1, 1, 1, 1, 1]

```

```
Process coin 2:
  sum 2: dp[2] += dp[2-2] = dp[0] = 1  →  [1, 1, 2, 1, 1]
  sum 3: dp[3] += dp[3-2] = dp[1] = 1  →  [1, 1, 2, 2, 1]
  sum 4: dp[4] += dp[4-2] = dp[2] = 2  →  [1, 1, 2, 2, 3]

Answer: dp[4] = 3
The three ways: [1,1,1,1], [1,1,2], [2,2]
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, target;
    cin >> n >> target;

    vector<int> coins(n);
    for (int i = 0; i < n; i++) {
        cin >> coins[i];
    }

    // dp[i] = number of unordered ways to form sum i
    vector<int> dp(target + 1, 0);
    dp[0] = 1;  // Base case: one way to make sum 0

    // Process each coin in order
    for (int coin : coins) {
        for (int sum = coin; sum <= target; sum++) {
            dp[sum] = (dp[sum] + dp[sum - coin]) % MOD;
        }
    }

    cout << dp[target] << '\n';
    return 0;
}
```

1.5. Removing Digits

Question - Removing Digits Backup Link

Explanation :

We start with a number and can repeatedly subtract any of its digits until we reach 0. The goal is to find the minimum number of steps required.

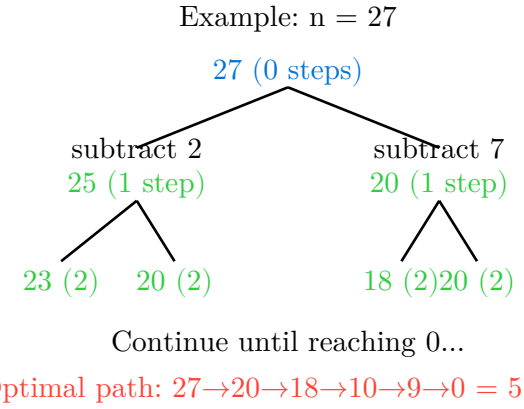
The Greedy Intuition (That Works!): At each step, we want to make the largest possible reduction. This means subtracting the largest digit. Intuitively, this seems optimal - why take 10 steps subtracting 1 when we could take fewer steps using larger digits?

Why Greedy Works: Every move reduces the number, and we can prove that greedily choosing the maximum digit never prevents us from reaching 0 or forces extra steps. However, for completeness and to match the DP pattern of the problemset, we'll use dynamic programming which naturally handles all cases.

Dynamic Programming Approach: For each number `n`, we try subtracting each of its digits and take the minimum. The recurrence is:

```
dp[n] = 1 + min(dp[n - d] for all digits d in n)
```

Building Bottom-Up: Start with `dp[0] = 0`. For each number from 1 to target, extract all its digits, try subtracting each one, and take the minimum steps needed.



Step-by-Step Example for n=27:

```
dp[0] = 0

dp[1] = 1 + dp[1-1] = 1 + dp[0] = 1
dp[2] = 1 + dp[2-2] = 1 + dp[0] = 1
...
dp[9] = 1 + dp[9-9] = 1 + dp[0] = 1

dp[10] = 1 + min(dp[10-1], dp[10-0])
       = 1 + min(dp[9], dp[10])
```

```
10        = 1 + dp[9] = 2
11
12  ...continuing...
13
14  dp[27] = 1 + min(dp[27-2], dp[27-7])
15         = 1 + min(dp[25], dp[20])
```

The answer builds up from smaller numbers to our target.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    // dp[i] = minimum steps to reduce i to 0
    vector<int> dp(n + 1, 1e9);
    dp[0] = 0;  // Base case: 0 needs 0 steps

    for (int i = 1; i <= n; i++) {
        // Extract all digits of i
        int temp = i;
        while (temp > 0) {
            int digit = temp % 10;
            if (digit > 0) {
                dp[i] = min(dp[i], dp[i - digit] + 1);
            }
            temp /= 10;
        }
    }

    cout << dp[n] << '\n';
    return 0;
}
```

1.6. Grid Paths I

Question - Grid Paths I Backup Link

Explanation :

We have an `n × n` grid and need to count paths from the top-left corner to the bottom-right corner, moving only right or down. Some cells may be blocked (traps).

The Core Idea: To reach any cell, we must come from either the cell above it or the cell to its left. The number of ways to reach a cell is the sum of ways to reach these two neighbors.

Recurrence Relation:

```
dp[i][j] = dp[i-1][j] + dp[i][j-1]
```

But we must check: if cell `(i,j)` is a trap, then `dp[i][j] = 0`.

Base Cases:

- `dp[0][0] = 1` if the starting cell is not a trap
- First row: can only reach from the left
- First column: can only reach from above

Edge Cases: If the starting or ending cell is a trap, the answer is 0.

Example: 4×4 grid with one trap at (1,2)

1	1	1	1
1	2	X	1
1	3	3	4
1	4	7	11

Visual Trace for 3×3 grid with no traps:

```
Initial grid:     After filling:
. . .             1  1  1
. . .       →     1  2  3
. . .             1  3  6

dp[0][0] = 1 (start)
dp[0][1] = 1 (can only go right from dp[0][0])
dp[0][2] = 1 (can only go right)

dp[1][0] = 1 (can only go down from dp[0][0])
dp[1][1] = dp[0][1] + dp[1][0] = 1 + 1 = 2
dp[1][2] = dp[0][2] + dp[1][1] = 1 + 2 = 3
```

```
13
14 dp[2][0] = 1
15 dp[2][1] = dp[1][1] + dp[2][0] = 2 + 1 = 3
16 dp[2][2] = dp[1][2] + dp[2][1] = 3 + 3 = 6
17
18 Answer: 6 paths
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<string> grid(n);
    for (int i = 0; i < n; i++) {
        cin >> grid[i];
    }

    // If start or end is blocked, answer is 0
    if (grid[0][0] == '*' || grid[n-1][n-1] == '*') {
        cout << 0 << '\n';
        return 0;
    }

    // dp[i][j] = number of paths to reach cell (i,j)
    vector<vector<int>> dp(n, vector<int>(n, 0));
    dp[0][0] = 1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == '*') {
                dp[i][j] = 0;
                continue;
            }

```

```
35              if (i > 0) {
36                  dp[i][j] = (dp[i][j] + dp[i-1][j]) % MOD;
37              }
38              if (j > 0) {
39                  dp[i][j] = (dp[i][j] + dp[i][j-1]) % MOD;
40              }
41          }
42      }
43
44      cout << dp[n-1][n-1] << '\n';
45      return 0;
46  }
```

1.7. Book Shop

Question - Book Shop Backup Link

Explanation :

This is the classic 0/1 Knapsack problem. We have `n` books, each with a price and number of pages. Given a maximum budget, we want to maximize the total pages we can buy. Each book can be bought at most once.

Why Not Greedy? We might think: "Buy books with the best pages-per-price ratio first." But this fails. Consider books with (price, pages): `[(3, 10), (4, 13), (5, 16)]` and budget 7. Greedy picks the first book (ratio 3.33), then can't fit anything else for 10 pages total. Optimal: pick the second book for 13 pages.

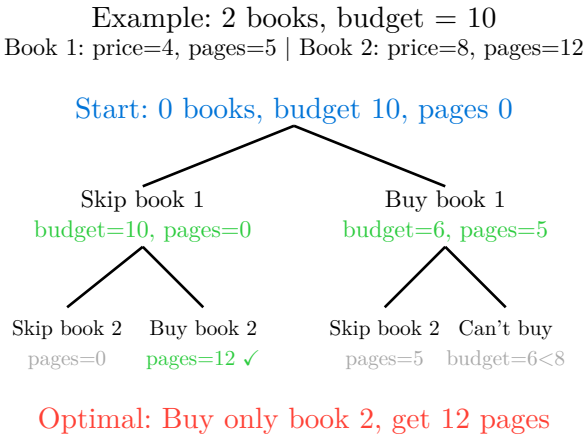
The DP Insight: For each book, we have two choices: buy it or skip it. We explore both options and take the maximum. The key is processing books one by one and tracking the best value achievable for each budget amount.

State Definition: `dp[i][budget]` = maximum pages achievable using first `i` books with exactly `budget` money.

Recurrence:

```
dp[i][budget] = max(
    dp[i-1][budget],                          // Don't buy book i
    dp[i-1][budget - price[i]] + pages[i]     // Buy book i
)
```

Space Optimization: We can optimize from 2D to 1D by processing budget in reverse order. This ensures we don't use the same book twice.



Step-by-Step Example:

```
Books: [(4,5), (8,12)]
Budget: 10

```

```
Initial: dp = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
         indices: 0  1  2  3  4  5  6  7  8  9  10

After book 1 (price=4, pages=5):
Process budget 10 down to 4:
  budget 10: dp[10] = max(dp[10], dp[10-4]+5) = max(0, 0+5) = 5
  budget 9: dp[9] = max(0, 0+5) = 5
  ...
  budget 4: dp[4] = max(0, 0+5) = 5

dp = [0, 0, 0, 0, 5, 5, 5, 5, 5, 5, 5]

After book 2 (price=8, pages=12):
  budget 10: dp[10] = max(dp[10], dp[10-8]+12) = max(5, 0+12) = 12
  budget 9: dp[9] = max(5, 0+12) = 12
  budget 8: dp[8] = max(5, 0+12) = 12

dp = [0, 0, 0, 0, 5, 5, 5, 5, 12, 12, 12]

Answer: dp[10] = 12
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, x;
    cin >> n >> x;

    vector<int> price(n), pages(n);
    for (int i = 0; i < n; i++) {
        cin >> price[i];
    }
    for (int i = 0; i < n; i++) {
        cin >> pages[i];
    }

    // dp[budget] = maximum pages with this budget
    vector<int> dp(x + 1, 0);
```

```
21
22      for (int i = 0; i < n; i++) {
23          // Process in reverse to avoid using same book twice
24          for (int budget = x; budget >= price[i]; budget--) {
25              dp[budget] = max(dp[budget], dp[budget - price[i]] + pages[i]);
26          }
27      }
28
29      cout << dp[x] << '\n';
30      return 0;
31  }
```

1.8. Array Description

Question - Array Description Backup Link

Explanation :

We have an array where some positions are fixed and others are marked with 0 (can be any value from 1 to m). Adjacent elements must differ by at most 1. Count the number of valid ways to fill in the zeros.

Key Constraint: For any two adjacent elements `a[i]` and `a[i+1]`, we need `|a[i] - a[i+1]| ≤ 1`. This means each element can only transition to values within ± 1 of its current value.

DP State: `dp[i][v]` = number of ways to fill positions 0 to i such that position i has value v.

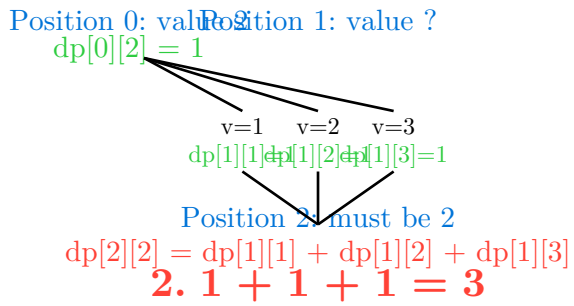
Transitions: For position i with value v, we can transition from position i-1 with values (v-1), v, or (v+1):

```
dp[i][v] = dp[i-1][v-1] + dp[i-1][v] + dp[i-1][v+1]
```

Handling Fixed Values:
- If `a[i]` is not 0, we can only use `dp[i][a[i]]`
- If `a[i]` is 0, we try all values from 1 to m

Edge Cases:
- If two consecutive fixed values differ by more than 1, answer is 0
- First position: if fixed, start with that value; otherwise try all 1 to m

Example: array = [2, 0, 2], m = 3



Valid arrays: [2,1,2], [2,2,2], [2,3,2]

Detailed Example:

```
Array: [0, 0, 0], m = 2

Position 0:
  dp[0][1] = 1
  dp[0][2] = 1

Position 1:
  dp[1][1] = dp[0][1] + dp[0][2] = 1 + 1 = 2
```

```
  dp[1][2] = dp[0][1] + dp[0][2] = 1 + 1 = 2

Position 2:
  dp[2][1] = dp[1][1] + dp[1][2] = 2 + 2 = 4
  dp[2][2] = dp[1][1] + dp[1][2] = 2 + 2 = 4

Answer: dp[2][1] + dp[2][2] = 4 + 4 = 8
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<int> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Check if consecutive fixed values are compatible
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] != 0 && arr[i + 1] != 0) {
            if (abs(arr[i] - arr[i + 1]) > 1) {
                cout << 0 << '\n';
                return 0;
            }
        }
    }

    // dp[i][v] = ways to fill up to position i with value v at position i
    vector<vector<long long>> dp(n, vector<long long>(m + 1, 0));

    // Initialize first position
    if (arr[0] == 0) {
        for (int v = 1; v <= m; v++) {
```

```
            dp[0][v] = 1;
        }
    } else {
        dp[0][arr[0]] = 1;
    }

    // Fill remaining positions
    for (int i = 1; i < n; i++) {
        if (arr[i] == 0) {
            // Try all values
            for (int v = 1; v <= m; v++) {
                for (int prev = max(1, v - 1); prev <= min(m, v + 1); prev++)
                {
                    dp[i][v] = (dp[i][v] + dp[i - 1][prev]) % MOD;
                }
            }
        } else {
            // Fixed value
            int v = arr[i];
            for (int prev = max(1, v - 1); prev <= min(m, v + 1); prev++) {
                dp[i][v] = (dp[i][v] + dp[i - 1][prev]) % MOD;
            }
        }
    }

    // Sum all valid endings
    long long result = 0;
    for (int v = 1; v <= m; v++) {
        result = (result + dp[n - 1][v]) % MOD;
    }

    cout << result << '\n';
    return 0;
}
```

2.1. Counting Towers

Question - Counting Towers Backup Link

Explanation :

Build a tower of width 2 and height n using 1×2 and 2×1 tiles. Count the number of distinct towers.

This is a complex DP problem involving state tracking. The key is to track whether the previous row had a vertical split or was filled horizontally.

State Definition:

- `dp[i][0]` = ways to fill up to row i with row i having two separate 1×1 blocks
- `dp[i][1]` = ways to fill up to row i with row i being a single 2×1 block

The transitions depend on how we can extend from the previous row, considering all valid placements.

Note: This is an advanced problem requiring careful case analysis. The solution involves tracking 4 different states based on the configuration of the top row.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;

    // Precompute for all possible heights
    const int MAXN = 1000001;
    vector<long long> dp0(MAXN), dp1(MAXN);

    dp0[1] = 1;
    dp1[1] = 1;

    for (int i = 2; i < MAXN; i++) {
        dp0[i] = (4 * dp0[i-1] + dp1[i-1]) % MOD;
        dp1[i] = (dp0[i-1] + 2 * dp1[i-1]) % MOD;
    }

```

```
25      while (t--) {
26          int n;
27          cin >> n;
28          cout << (dp0[n] + dp1[n]) % MOD << '\n';
29      }
30
31      return 0;
32  }
```

2.2. Edit Distance

Question - Edit Distance Backup Link

Explanation :

Given two strings, find the minimum number of operations (insert, delete, or replace a character) needed to transform one string into the other. This is the classic Levenshtein distance problem.

The Intuition: Compare strings character by character from left to right. At each position, we have choices based on whether characters match or differ.

State Definition: `dp[i][j]` = minimum operations to transform the first `i` characters of string 1 into the first `j` characters of string 2.

The Three Operations:
1. **Replace**: Change `s1[i]` to `s2[j]`, then solve for remaining strings
2. **Insert**: Add `s2[j]` to `s1`, then match it against `s2[j]`
3. **Delete**: Remove `s1[i]` from `s1`

Recurrence:

```
If s1[i] == s2[j]:
  dp[i][j] = dp[i-1][j-1]  // No operation needed

Otherwise:
  dp[i][j] = 1 + min(
    dp[i-1][j-1],  // Replace
    dp[i][j-1],    // Insert
    dp[i-1][j]     // Delete
  )
```

Base Cases:
- `dp[0][j] = j` (insert j characters)
- `dp[i][0] = i` (delete i characters)

Example: "LOVE" → "MOVIE"

	ε	M	O	V	I	E
ε	0	1	2	3	4	5
L	1	1	2	3	4	5
O	2	2	1	2	3	4
V	3	3	2	1	2	3
E	4	4	3	2	2	2

Transformations needed:
Insert 'M' at start: LOVE → MLOVE
Replace 'L' with 'O': MLOVE → MOOVE
Delete 'O': MOOVE → MOVE
Insert 'I': MOVE → MOVIE
Minimum: 2 operations

Trace Example: "CAT" → "DOG"

```
    ε  D  O  G
ε   0  1  2  3
C   1  1  2  3
A   2  2  2  3
T   3  3  3  3

dp[3][3] = 3 operations needed
(Replace C→D, A→O, T→G)
```

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s1, s2;
    cin >> s1 >> s2;

    int n = s1.length();
    int m = s2.length();

    // dp[i][j] = min operations to transform s1[0..i-1] to s2[0..j-1]
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    // Base cases
    for (int i = 0; i <= n; i++) {
        dp[i][0] = i;  // Delete all i characters
    }
    for (int j = 0; j <= m; j++) {
        dp[0][j] = j;  // Insert all j characters
    }

    // Fill the DP table
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                // Characters match, no operation needed
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                // Take minimum of three operations
```

```
                dp[i][j] = 1 + min({
                    dp[i - 1][j - 1],  // Replace
                    dp[i][j - 1],      // Insert
                    dp[i - 1][j]       // Delete
                });
            }
        }
    }

    cout << dp[n][m] << '\n';
    return 0;
}
```

2.3. Longest Common Subsequence

Question - Longest Common Subsequence Backup Link

Explanation :

Find the longest subsequence that appears in both strings. A subsequence is a sequence that can be derived by deleting some (or no) characters without changing the order of remaining characters.

Example: "ABCD" and "AEBD" have LCS "ABD" (length 3).

Key Distinction:
- **Subsequence**: Characters don't need to be consecutive (can skip characters)
- **Substring**: Characters must be consecutive

The Intuition: Build the solution by comparing characters from both strings. When characters match, we extend the common subsequence. When they don't match, we try both options: skip from string 1 or skip from string 2, and take the better result.

State Definition: `dp[i][j]` = length of LCS using first `i` characters of s1 and first `j` characters of s2.

Recurrence:

```
If s1[i-1] == s2[j-1]:
  dp[i][j] = 1 + dp[i-1][j-1]  // Match found, extend LCS

Otherwise:
  dp[i][j] = max(
    dp[i-1][j],   // Skip character from s1
    dp[i][j-1]    // Skip character from s2
  )
```

Base Cases:
- `dp[0][j] = 0` (no characters from s1)
- `dp[i][0] = 0` (no characters from s2)

Example: "AGGTAB" and "GXTXAYB"

	ε	G	X	T	X	A	Y	B
ε	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
G	0	1	1	1	1	1	1	1
G	0	1	1	1	1	1	1	1
T	0	1	1	2	2	2	2	2
A	0	1	1	2	2	3	3	3
B	0	1	1	2	2	3	3	4

Matching characters build up the LCS:

G matches G
T matches T
A matches A
B matches B

LCS: "GTAB" (length 4)

Step-by-Step Trace: "ACE" and "ABCDE"

```
    ε  A  B  C  D  E
ε   0  0  0  0  0  0
A   0  1  1  1  1  1   (A matches A)
C   0  1  1  2  2  2   (C matches C)
E   0  1  1  2  2  3   (E matches E)

LCS = "ACE" with length 3
```

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s1, s2;
    cin >> s1 >> s2;

    int n = s1.length();
    int m = s2.length();

    // dp[i][j] = LCS length using s1[0..i-1] and s2[0..j-1]
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                // Characters match, extend LCS
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                // Take maximum of skipping from either string
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    cout << dp[n][m] << '\n';
    return 0;
}
```

2.4. Rectangle Cutting

Question - Rectangle Cutting Backup Link

Explanation :

Given an `a × b` rectangle, find the minimum number of cuts needed to divide it into squares. Each cut must be parallel to a side and go through the entire piece.

Base Cases:
- If the rectangle is already a square (`a == b`), we need 0 cuts
- Otherwise, we need at least 1 cut

The Strategy: We can make either horizontal or vertical cuts. Try all possible cut positions and recursively solve for the resulting pieces. Choose the cut that minimizes total operations.

State Definition: `dp[h][w]` = minimum cuts needed for an `h × w` rectangle.

Recurrence:

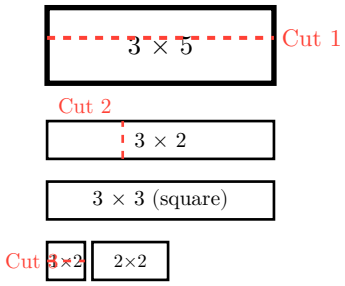
```
If h == w: dp[h][w] = 0  (already a square)

Otherwise:
  Try horizontal cuts at each row i:
    dp[h][w] = min(dp[h][w], dp[i][w] + dp[h-i][w] + 1)

  Try vertical cuts at each column j:
    dp[h][w] = min(dp[h][w], dp[h][j] + dp[h][w-j] + 1)
```

Optimization Note: Due to symmetry, `dp[h][w] = dp[w][h]`.

Example: 3×5 rectangle



Total: 3 cuts for 3×5 rectangle

Example Trace for 2×3:

```
dp[1][1] = 0 (square)
dp[1][2] = dp[1][1] + dp[1][1] + 1 = 0 + 0 + 1 = 1
dp[1][3] = dp[1][1] + dp[1][2] + 1 = 0 + 1 + 1 = 2
dp[2][2] = 0 (square)
dp[2][3] = min(
```

```
6    dp[1][3] + dp[1][3] + 1 = 2 + 2 + 1 = 5,
7    dp[2][1] + dp[2][2] + 1 = 1 + 0 + 1 = 2
8  ) = 2
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int a, b;
    cin >> a >> b;

    // dp[i][j] = minimum cuts for i×j rectangle
    vector<vector<int>> dp(a + 1, vector<int>(b + 1, 1e9));

    // Base case: squares need 0 cuts
    for (int i = 1; i <= min(a, b); i++) {
        dp[i][i] = 0;
    }

    for (int h = 1; h <= a; h++) {
        for (int w = 1; w <= b; w++) {
            if (h == w) continue;  // Already a square

            // Try horizontal cuts
            for (int i = 1; i < h; i++) {
                dp[h][w] = min(dp[h][w], dp[i][w] + dp[h - i][w] + 1);
            }

            // Try vertical cuts
            for (int j = 1; j < w; j++) {
                dp[h][w] = min(dp[h][w], dp[h][j] + dp[h][w - j] + 1);
            }
        }
    }

    cout << dp[a][b] << '\n';
    return 0;
}
```

2.5. Minimal Grid Path

Question - Minimal Grid Path Backup Link

Explanation :

This is an advanced optimization problem involving grid paths with varying costs for horizontal and vertical moves.

Note: This problem requires careful mathematical optimization and is beyond the scope of standard DP patterns. It involves prefix minimums and mathematical analysis.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<long long> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    long long ans = 1e18;
    long long sum = 0;
    long long min_odd = 1e18, min_even = 1e18;

    for (int i = 0; i < n; i++) {
        sum += a[i];
        if (i % 2 == 0) {
            min_even = min(min_even, a[i]);
            ans = min(ans, sum + min_odd * (n - i - 1));
        } else {
            min_odd = min(min_odd, a[i]);
            ans = min(ans, sum + min_even * (n - i - 1));
        }
    }

    cout << ans << '\n';
```

```
32      return 0;
33  }
```

2.6. Money Sums

Question - Money Sums Backup Link

Explanation :

Given `n` coins with specific values, find all distinct sums you can form using any subset of these coins.

This is a Subset Sum Variation: Instead of checking if a specific sum is possible, we need to find ALL possible sums. The approach is similar to the 0/1 knapsack pattern.

The Key Insight: Use a boolean array to track which sums are achievable. For each coin, update all sums that become reachable by including that coin.

State Definition: `dp[sum]` = true if we can form this sum, false otherwise.

Algorithm:
1. Start with `dp[0] = true` (sum 0 is always achievable by taking no coins)
2. For each coin with value `c`:
   - Process sums in reverse order (to avoid using the same coin twice)
   - For each achievable sum `s`, mark `s + c` as achievable
3. Collect all sums where `dp[sum] = true`

Why Process in Reverse? When updating from left to right, we might use the same coin multiple times in one iteration. Processing right to left ensures each coin is considered only once.

Example: coins = [2, 3, 5]

Initially:
dp[0] = true

After coin 2:
dp[0] = true, dp[2] = true
$0 \to +2 \to 2$

After coin 3:
dp[0], dp[2], dp[3], dp[5] = true
$0 \to +3 \to 3$
$2 \to +3 \to 5$

After coin 5:
dp[0], dp[2], dp[3], dp[5], dp[7], dp[8], dp[10] = true
$0 \to +5 \to 5$
$2 \to +5 \to 7$
$3 \to +5 \to 8$
$5 \to +5 \to 10$

Possible sums: 2, 3, 5, 7, 8, 10

Step-by-Step Trace:

```
Coins: [1, 3]

Initial: dp[0] = true

Process coin 1:
  dp[1] = dp[1] | dp[0] = true
  Result: [T, T, F, F]

```

```
Process coin 3:
  dp[4] = dp[4] | dp[1] = true
  dp[3] = dp[3] | dp[0] = true
  Result: [T, T, F, T, T]

Possible sums: 1, 3, 4
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<int> coins(n);
    int total = 0;
    for (int i = 0; i < n; i++) {
        cin >> coins[i];
        total += coins[i];
    }

    // dp[sum] = true if sum is achievable
    vector<bool> dp(total + 1, false);
    dp[0] = true;  // Base case: sum 0 is always achievable

    for (int coin : coins) {
        // Process in reverse to avoid using same coin twice
        for (int sum = total; sum >= coin; sum--) {
            if (dp[sum - coin]) {
                dp[sum] = true;
            }
        }
    }

    // Collect all achievable sums (excluding 0)
    vector<int> result;
    for (int sum = 1; sum <= total; sum++) {
        if (dp[sum]) {
```

```
35              result.push_back(sum);
36          }
37      }
38
39      cout << result.size() << '\n';
40      for (int sum : result) {
41          cout << sum << ' ';
42      }
43      cout << '\n';
44
45      return 0;
46  }
```

2.7. Removal Game

Question - Removal Game Backup Link

Explanation :

Two players take turns removing numbers from either end of a list. Each player wants to maximize their own score. Both players play optimally. Find the score difference (player 1′s score minus player 2′s score) when player 1 goes first.

Key Insight - Minimax Strategy: Since both players play optimally, when it's your turn, you choose the move that maximizes your advantage. When it's the opponent's turn, they minimize your advantage (equivalently, maximize their own).

State Definition: `dp[l][r]` = maximum score advantage (your score - opponent's score) you can get from the subarray from index `l` to `r` when it's your turn.

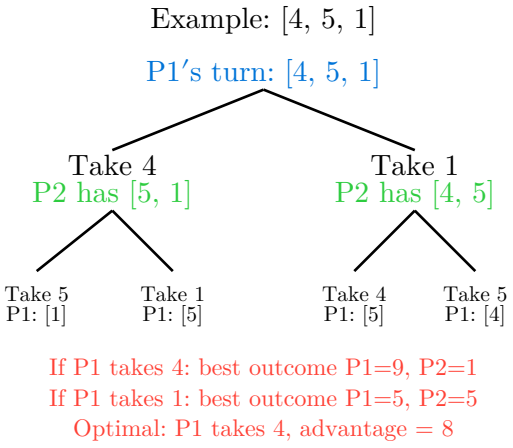
The Recurrence: When it's your turn on range `[l, r]`, you have two choices:

1. Take left element `arr[l]`: You get `arr[l]` points, opponent plays on `[l+1, r]`
   - Your advantage = `arr[l] - dp[l+1][r]`
2. Take right element `arr[r]`: You get `arr[r]` points, opponent plays on `[l, r-1]`
   - Your advantage = `arr[r] - dp[l][r-1]`

You choose the option that maximizes your advantage:

```
dp[l][r] = max(arr[l] - dp[l+1][r], arr[r] - dp[l][r-1])
```

Why Subtract Opponent's DP? The opponent's `dp` value represents their advantage over you in their turn. Subtracting it accounts for the points they'll gain relative to you.



Step-by-Step Example: [2, 5, 1]

```
Base cases (single elements):
dp[0][0] = 2  (just take it)
dp[1][1] = 5
dp[2][2] = 1

```

```
Two elements:
dp[0][1] = max(2 - dp[1][1], 5 - dp[0][0])
         = max(2 - 5, 5 - 2) = max(-3, 3) = 3

dp[1][2] = max(5 - dp[2][2], 1 - dp[1][1])
         = max(5 - 1, 1 - 5) = max(4, -4) = 4

Three elements:
dp[0][2] = max(2 - dp[1][2], 1 - dp[0][1])
         = max(2 - 4, 1 - 3) = max(-2, -2) = -2

Wait, this gives -2, meaning P2 wins by 2!
Actual: P1=3, P2=5, so P1's advantage is 3-5 = -2 ✓
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<long long> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // dp[l][r] = max advantage (my score - opponent score) for range [l,r]
    vector<vector<long long>> dp(n, vector<long long>(n, 0));

    // Base case: single elements
    for (int i = 0; i < n; i++) {
        dp[i][i] = arr[i];
    }

    // Fill for increasing lengths
    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;
```

```
            dp[l][r] = max(
                arr[l] - dp[l + 1][r],   // Take left
                arr[r] - dp[l][r - 1]    // Take right
            );
        }
    }

    // Convert advantage to player 1's actual score
    long long total = 0;
    for (long long x : arr) {
        total += x;
    }

    long long player1_score = (total + dp[0][n - 1]) / 2;
    cout << player1_score << '\n';

    return 0;
}
```

2.8. Two Sets II

Question - Two Sets II Backup Link

Explanation :

Divide numbers 1 to n into two sets with equal sums. Count the number of ways to do this.

First Observation: The sum of 1 to n is `n * (n+1) / 2`. For two equal sets, this sum must be even, and each set must have sum `total / 2`.

Key Insight: This reduces to a subset sum counting problem: "In how many ways can we select numbers from 1 to n to form sum `total / 2`?"

Important Symmetry: If we count all ways to form one set, we're also counting its complement. For example, if we select {1, 4} for sum 5, we automatically get {2, 3} as the other set. So we need to divide by 2 to avoid counting each partition twice.

State Definition: `dp[sum]` = number of ways to form exactly this sum.

Algorithm:
1. Check if `total` is even; if not, return 0
2. Use subset sum DP to count ways to form `target = total / 2`
3. Divide result by 2 (due to symmetry)

Example: n = 4
Numbers: 1, 2, 3, 4 Total sum: 10 Target per set: 5

Set 1		Set 2
{1, 4}		{2, 3}
{2, 3}		{1, 4}

These are the same partition!

Ways to make sum 5: {1,4} and {2,3}
Count = 2, but only 1 unique partition
Answer: 2 / 2 = 1

Trace for n = 4:

```
Target sum = 10/2 = 5

dp[0] = 1

Process 1: dp[1] = 1
Process 2: dp[3] = 1, dp[2] = 1
Process 3: dp[6] = 1, dp[5] = 1, dp[4] = 1, dp[3] = 2
Process 4: dp[9] = 1, dp[8] = 1, dp[7] = 1, dp[6] = 2, dp[5] = 2, dp[4] = 2

dp[5] = 2
Answer = 2 / 2 = 1
```

Edge Case: When n = 1 or sum is odd, answer is 0.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    long long total = (long long)n * (n + 1) / 2;

    // If total sum is odd, can't divide into two equal sets
    if (total % 2 != 0) {
        cout << 0 << '\n';
        return 0;
    }

    int target = total / 2;

    // dp[sum] = number of ways to form this sum
    vector<long long> dp(target + 1, 0);
    dp[0] = 1;

    for (int num = 1; num <= n; num++) {
        for (int sum = target; sum >= num; sum--) {
            dp[sum] = (dp[sum] + dp[sum - num]) % MOD;
        }
    }

    // Divide by 2 because each partition is counted twice
    // Need modular inverse of 2, which is (MOD+1)/2
    long long result = (dp[target] * ((MOD + 1) / 2)) % MOD;
    cout << result << '\n';

    return 0;
}
```

2.9. Mountain Range

Question - Mountain Range Backup Link

Explanation :

Count valid mountain ranges of length 2n (sequences of n up-steps and n down-steps that never go below the starting level). This is equivalent to counting Catalan numbers.

State Definition: `dp[steps][height]` = number of ways to place `steps` moves and be at `height`.

The nth Catalan number formula: C(n) = C(2n, n) / (n+1)

Note: This involves Catalan number computation with modular arithmetic.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

long long power(long long a, long long b, long long mod) {
    long long res = 1;
    while (b > 0) {
        if (b & 1) res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    // Compute C(2n, n) / (n+1) using modular arithmetic
    vector<long long> fact(2 * n + 1);
    fact[0] = 1;
    for (int i = 1; i <= 2 * n; i++) {
        fact[i] = (fact[i - 1] * i) % MOD;
    }

```

```
    long long numerator = fact[2 * n];
    long long denominator = (fact[n] * fact[n]) % MOD;
    denominator = (denominator * (n + 1)) % MOD;

    long long result = (numerator * power(denominator, MOD - 2, MOD)) % MOD;
    cout << result << '\n';

    return 0;
}
```

2.10. Increasing Subsequence

Question - Increasing Subsequence Backup Link

Explanation :

Find the length of the longest strictly increasing subsequence (LIS) in an array. This is a classic problem with an efficient O(n log n) solution.

Naive DP Approach - $O(n^2)$: For each position i, check all previous positions $j < i$. If `arr[j] < arr[i]`, we can extend that subsequence. This works but is too slow for large n.

Efficient Approach - O(n log n): Maintain an array `tails` where `tails[len]` stores the smallest ending value of all increasing subsequences of length `len+1`.

Key Insight: If we want to build longer subsequences, we should keep the smallest possible ending values. This gives us more room to extend later.

Algorithm:
1. For each element, find where it can be placed in `tails` using binary search
2. If element is larger than all tails, append it (new longer subsequence)
3. Otherwise, replace the first tail that is >= element (maintain smallest endings)

Why This Works: The `tails` array is always sorted. When we replace a tail, we're saying "there's now a better (smaller) ending for this length, which gives more potential for extension."

Example: [6, 2, 5, 1, 7, 4, 8, 3]

Process 6	tails: \[6\]	LIS length: 1
Process 2	tails: \[2\]	Replace 6 with 2
Process 5	tails: \[2, 5\]	LIS length: 2
Process 1	tails: \[1, 5\]	Replace 2 with 1
Process 7	tails: \[1, 5, 7\]	LIS length: 3
Process 4	tails: \[1, 4, 7\]	Replace 5 with 4
Process 8	tails: \[1, 4, 7, 8\]	LIS length: 4
Process 3	tails: \[1, 3, 7, 8\]	Replace 4 with 3

Final answer: 4

Why Binary Search? We need to find the leftmost position in `tails` where we can place the current element. Since `tails` is sorted, binary search gives us O(log n) per element.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```cpp

    int n;
    cin >> n;

    vector<int> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // tails[i] = smallest ending value of all LIS of length i+1
    vector<int> tails;

    for (int i = 0; i < n; i++) {
        // Find position where arr[i] can be placed
        auto pos = lower_bound(tails.begin(), tails.end(), arr[i]);

        if (pos == tails.end()) {
            // arr[i] is larger than all tails, extend LIS
            tails.push_back(arr[i]);
        } else {
            // Replace to maintain smallest ending value
            *pos = arr[i];
        }
    }

    cout << tails.size() << '\n';
    return 0;
}
```

2.11. Projects

Question - Projects Backup Link

Explanation :

We have n projects, each with a start day, end day, and reward. We can only work on one project at a time (no overlapping). Find the maximum total reward.

This is the Weighted Job Scheduling problem, a variation of the activity selection problem.

Key Insight: Sort projects by ending time. For each project, decide whether to include it or skip it. If we include it, we need to find the latest non-overlapping project that ended before this one starts.

State Definition: `dp[i]` = maximum reward using projects from 0 to i (where projects are sorted by end time).

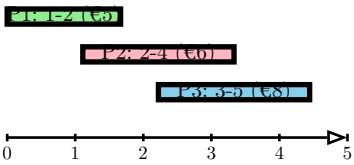
Recurrence:

```
dp[i] = max(
  dp[i-1],                          // Skip project i
  projects[i].reward + dp[last]     // Include project i
)

where last = largest index where projects[last].end < projects[i].start
```

Finding the Last Non-Overlapping Project: Use binary search to find the latest project that ends before the current project starts. This makes the solution O(n log n).

Example: 3 projects



Sorted by end time: P1, P2, P3

Decision tree:

Skip P1: €0 Take P2 (after skip): €6
Take P1: €5 Take P3 (after P1): €5+€8=€13
Take P3 (after skip): €8

Optimal: Take P1 and P3 = €13

Algorithm Steps:

```
1. Sort projects by end time
2. For each project, use binary search to find last non-overlapping
3. Compute: dp[i] = max(skip, take)
4. Return dp[n-1]
```

Example Trace:

```
Projects (sorted by end): [(1,2,5), (2,4,6), (3,5,8)]

dp[0] = 5 (take first project)

dp[1]: last = -1 (no non-overlapping before index 0)
       max(dp[0], 6 + 0) = max(5, 6) = 6

dp[2]: last = 0 (project 0 ends at 2, project 2 starts at 3)
       max(dp[1], 8 + dp[0]) = max(6, 8 + 5) = 13

Answer: 13
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Project {
    int start, end;
    long long reward;
    bool operator<(const Project& other) const {
        return end < other.end;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<Project> projects(n);
    for (int i = 0; i < n; i++) {
        cin >> projects[i].start >> projects[i].end >> projects[i].reward;
    }

    // Sort by end time
    sort(projects.begin(), projects.end());

    // dp[i] = max reward using projects 0 to i
    vector<long long> dp(n);
    dp[0] = projects[0].reward;
```

```cpp

    for (int i = 1; i < n; i++) {
        // Option 1: Skip current project
        long long skip = dp[i - 1];

        // Option 2: Take current project
        // Find last non-overlapping project using binary search
        int left = 0, right = i - 1, last = -1;
        while (left <= right) {
            int mid = (left + right) / 2;
            if (projects[mid].end < projects[i].start) {
                last = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        long long take = projects[i].reward;
        if (last != -1) {
            take += dp[last];
        }

        dp[i] = max(skip, take);
    }

    cout << dp[n - 1] << '\n';
    return 0;
}
```

2.12. Elevator Rides

Question - Elevator Rides Backup Link

Explanation :

We have n people with known weights and an elevator with maximum capacity x. Find the minimum number of elevator rides needed to transport everyone.

This is a Bitmask DP problem where we optimize over subsets of people.

Key Insight: For any subset of people, we want to know: "What's the minimum number of rides needed, and how much weight is used in the last ride?" This lets us extend the solution by adding more people.

State Definition: `dp[mask]` = pair (rides, weight) representing:
- Minimum rides needed for this subset
- Weight used in the last ride

Why Track Last Ride Weight? When adding a new person to a subset, we need to know if they can fit in the current ride or need a new one.

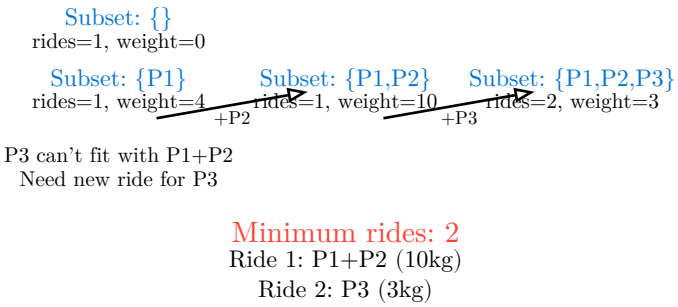
Transition: For each subset `mask` and each person `i` not in mask:

```
new_mask = mask | (1 << i)

If weight + person[i] <= capacity:
  // Person fits in current ride
  dp[new_mask] = (rides, weight + person[i])
Else:
  // Need a new ride
  dp[new_mask] = (rides + 1, person[i])
```

Complexity: O(2^n × n), which works for n ≤ 20.

Example: 3 people, weights [4, 6, 3], capacity = 10



Bitmask Representation:

```
Empty set: 0000 (binary) = 0
{P1}:      0001 (binary) = 1
{P2}:      0010 (binary) = 2
```

```
4 {P1,P2}:   0011 (binary) = 3
5 {P1,P2,P3}: 0111 (binary) = 7
```

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, x;
    cin >> n >> x;

    vector<int> weight(n);
    for (int i = 0; i < n; i++) {
        cin >> weight[i];
    }

    // dp[mask] = {rides, weight in last ride}
    vector<pair<int, int>> dp(1 << n);
    dp[0] = {1, 0};  // Base case: 0 people, 1 ride (empty), 0 weight

    for (int mask = 1; mask < (1 << n); mask++) {
        dp[mask] = {n + 1, 0};  // Initialize with worst case

        for (int i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                // Person i is in this subset
                int prev = mask ^ (1 << i);  // Remove person i
                auto [rides, last_weight] = dp[prev];

                if (last_weight + weight[i] <= x) {
                    // Person i fits in current ride
                    dp[mask] = min(dp[mask], {rides, last_weight +
                    weight[i]});
                } else {
                    // Need a new ride for person i
                    dp[mask] = min(dp[mask], {rides + 1, weight[i]});
                }
            }
        }
    }
```

```
39
40      cout << dp[(1 << n) - 1].first << '\n';
41      return 0;
42  }
```

2.13. Counting Tilings

Question - Counting Tilings Backup Link

Explanation :

Fill an n×m grid completely using 1×2 and 2×1 tiles. Count the number of ways.

This is a classic profile/broken profile dynamic programming problem. We process the grid column by column, tracking which cells in the current column are filled by tiles from the previous column.

State Definition: `dp[col][mask]` = number of ways to fill up to column col, where mask represents which cells in column col are already occupied.

The key insight is using bitmask to represent column states and carefully generating all valid transitions.

Note: This is an advanced bitmask DP problem. The solution involves generating valid mask transitions.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;
int n, m;
vector<long long> dp[1001];
vector<int> next_masks[1024];

void generate_next(int cur_mask, int next_mask, int col, int n) {
    if (col == n) {
        next_masks[cur_mask].push_back(next_mask);
        return;
    }
    if ((cur_mask >> col) & 1) {
        generate_next(cur_mask, next_mask, col + 1, n);
    } else {
        generate_next(cur_mask | (1 << col), next_mask | (1 << col), col + 1,
        n);
        if (col + 1 < n && !((cur_mask >> (col + 1)) & 1)) {
            generate_next(cur_mask | (1 << col) | (1 << (col + 1)),
            next_mask, col + 2, n);
        }
    }
}

```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;

    for (int mask = 0; mask < (1 << n); mask++) {
        generate_next(mask, 0, 0, n);
    }

    for (int i = 0; i <= m; i++) {
        dp[i].assign(1 << n, 0);
    }
    dp[0][0] = 1;

    for (int col = 0; col < m; col++) {
        for (int mask = 0; mask < (1 << n); mask++) {
            if (dp[col][mask] == 0) continue;
            for (int next_mask : next_masks[mask]) {
                dp[col + 1][next_mask] = (dp[col + 1][next_mask] + dp[col][mask]) % MOD;
            }
        }
    }

    cout << dp[m][0] << '\n';
    return 0;
}
```

2.14. Counting Numbers

Question - Counting Numbers Backup Link

Explanation :

Count numbers in range [a, b] that have no adjacent equal digits.

This is a digit DP problem. We build numbers digit by digit, tracking:
- Current position
- Previous digit
- Whether we're still bounded by the upper limit
- Whether we've started placing non-zero digits

The answer is `count(b) - count(a-1)`.

Note: This is an advanced digit DP problem requiring careful state management.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

long long dp[20][10][2][2];
string num;

long long solve(int pos, int prev, int tight, int started) {
    if (pos == num.size()) {
        return started;
    }

    if (dp[pos][prev][tight][started] != -1) {
        return dp[pos][prev][tight][started];
    }

    int limit = tight ? (num[pos] - '0') : 9;
    long long res = 0;

    for (int digit = 0; digit <= limit; digit++) {
        if (started && digit == prev) continue;
        res += solve(
            pos + 1,
            digit,
            tight && (digit == limit),
            started || (digit > 0)
        );
    }
```

```
28
29      return dp[pos][prev][tight][started] = res;
30  }
31
32  long long count(long long x) {
33      if (x < 0) return 0;
34      num = to_string(x);
35      memset(dp, -1, sizeof(dp));
36      return solve(0, 0, 1, 0);
37  }
38
39  int main() {
40      ios::sync_with_stdio(false);
41      cin.tie(nullptr);
42
43      long long a, b;
44      cin >> a >> b;
45
46      cout << count(b) - count(a - 1) << '\n';
47      return 0;
48  }
```

2.15. Increasing Subsequence II

Question - Increasing Subsequence II Backup Link

Explanation :

Count the number of increasing subsequences of length at least 1. This is different from the previous problem which asked for the length. Here we need to count ALL such subsequences.

Key Observation: For each position i, we need to know: "How many increasing subsequences end at position i?" The answer is the sum of all these values.

State Definition: `dp[i]` = number of increasing subsequences ending at element i.

Recurrence:

```
dp[i] = 1 + sum(dp[j] for all j < i where arr[j] < arr[i])
```

The "1" accounts for the subsequence containing only element i.

Optimization with Coordinate Compression and Segment Tree: The naive $O(n^2)$ approach is too slow. We can optimize to O(n log n) using a segment tree or Fenwick tree to efficiently query the sum of all dp values for elements smaller than current.

Coordinate Compression: Since values can be large (up to 10^9), we compress them to indices 0 to n-1.

Algorithm:
1. Compress coordinates
2. For each element (left to right):
   - Query sum of dp values for all smaller elements
   - Update dp[i] = 1 + query_result
   - Insert dp[i] into the data structure at compressed position

Example: [2, 5, 3, 7]

Process 2: dp[0] = 1 (just [2])

Process 5: dp[1] = 1 + dp[0] = 2
Subsequences: [5], [2,5]

Process 3: dp[2] = 1 + dp[0] = 2
Subsequences: [3], [2,3]

Process 7: dp[3] = 1 + (dp[0] + dp[1] + dp[2])
3. 1 + (1 + 2 + 2) = 6
New subsequences:
[7], [2,7], [2,5,7]
[2,3,7], [5,7], [3,7]

Total: 1 + 2 + 2 + 6 = 11 subsequences

Why This Works: When we process element i, all dp values for smaller elements j are already computed. By summing them and adding 1, we count all ways to extend previous subsequences plus the subsequence containing only element i.

Code :

```
#include <bits/stdc++.h>
```
C++

```cpp
using namespace std;

const int MOD = 1e9 + 7;

class FenwickTree {
public:
    vector<long long> tree;
    int n;

    FenwickTree(int n) : n(n), tree(n + 1, 0) {}

    void update(int idx, long long val) {
        for (++idx; idx <= n; idx += idx & -idx) {
            tree[idx] = (tree[idx] + val) % MOD;
        }
    }

    long long query(int idx) {
        long long sum = 0;
        for (++idx; idx > 0; idx -= idx & -idx) {
            sum = (sum + tree[idx]) % MOD;
        }
        return sum;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<int> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Coordinate compression
    vector<int> sorted = arr;
    sort(sorted.begin(), sorted.end());
    sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());

    map<int, int> compress;
    for (int i = 0; i < sorted.size(); i++) {
```

```
47          compress[sorted[i]] = i;
48      }
49
50      FenwickTree ft(sorted.size());
51      long long total = 0;
52
53      for (int i = 0; i < n; i++) {
54          int pos = compress[arr[i]];
55
56          // Query sum of all dp values for elements < arr[i]
57          long long prev_sum = 0;
58          if (pos > 0) {
59              prev_sum = ft.query(pos - 1);
60          }
61
62          long long dp_i = (1 + prev_sum) % MOD;
63          ft.update(pos, dp_i);
64
65          total = (total + dp_i) % MOD;
66      }
67
68      cout << total << '\n';
69      return 0;
70  }
```